

PipelineDesk CRM

Aplicación CRM Pipeline para centros de formación

Presentación del proyecto

PHP · MySQL · JavaScript · HTML · CSS · Tailwind

SERGIO PAVÓN DÍAZ

2º DAW

IES CRISTOBAL DEL MONROY



PipelineDesk

Puntos a tratar:

Recorrido desde el problema hasta las mejoras futuras

1

Objetivo y contexto del proyecto

2

Funcionalidades principales del CRM

3

Arquitectura MVC y modelo de datos

4

Implementación por fases

5

Dificultades encontradas

6

Cambios respecto a la planificación inicial

7

Propuestas de mejora y conclusiones

1 Objetivo del proyecto

Crear un CRM ligero para gestionar el ciclo comercial de un centro de formación: captar interesados, hacer seguimiento, moverlos por el embudo y analizar resultados.

- Captación desde formulario público
- Gestión de leads y estados
- Tareas, objeciones e histórico
- Dashboard comercial con objetivos

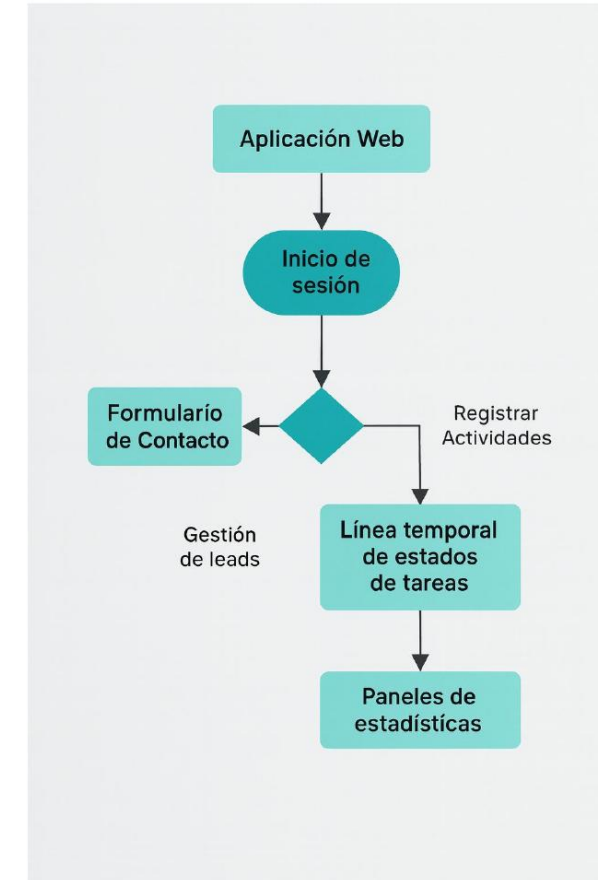


Figura 1. Diagrama inicial de los principales flujos de la aplicación.

Contexto y mercado objetivo

Una solución vertical frente a CRMs demasiado amplios

Necesidad detectada

- Academias pequeñas y medianas necesitan control comercial sin complejidad enterprise.
- PipelineDesk prioriza simplicidad, rapidez de implantación y adaptación a procesos educativos.
- El foco no es competir en amplitud, sino en adecuación al embudo de matriculación.
- Automatizar tareas en el departamento de Ventas.

2. Estudio de la competencia

El análisis de competencia se realizó para posicionar PipelineDesk dentro del mercado de herramientas CRM y detectar un hueco específico: soluciones ligeras, comprensibles y adaptadas a academias o centros de formación. Frente a plataformas generalistas o complejas, PipelineDesk se enfoca en captación, seguimiento comercial, embudo de ventas y operativa diaria del vendedor.

2.1 Clasificación de empresas del sector

Grupo	Descripción	Ejemplos	Lectura para PipelineDesk
CRM enterprise	Plataformas muy completas, escalables y con ecosistemas de integración amplios.	Salesforce	Sobredimensionadas para una academia pequeña o mediana.
CRM para pymes	Herramientas más económicas, centradas en ventas y marketing.	HubSpot CRM, Zoho CRM, Efficacy	Competidores directos por sencillez y coste.
CRM integrados en ERP	Módulos CRM conectados con contabilidad, RRHH o facturación.	Odoo CRM	Útiles para organizaciones que priorizan un todo-en-uno.
CRM verticales	Productos orientados a un sector con flujos concretos.	Aplicaciones educativas especializadas	Espacio donde PipelineDesk aporta mayor valor.

2.2 Necesidades reales del mercado objetivo

- Captación automática de leads desde formularios web o campañas.
- Clasificación rápida por curso o servicio de interés.
- Seguimiento comercial sencillo: llamadas, emails, notas y tareas.
- Pipeline visual para saber en qué etapa se encuentra cada oportunidad.
- Análisis básico del rendimiento comercial sin necesidad de herramientas complejas.
- Coste reducido y curva de aprendizaje baja para academias y pequeños equipos.

2.3 Comparativa de soluciones existentes

Aplicación	Fortalezas	Debilidades detectadas	Oportunidad para PipelineDesk
Salesforce	Muy potente, ecosistema amplio, automatización avanzada.	Coste alto y complejidad excesiva para una academia.	Ofrecer un CRM sencillo, vertical y rápido de implantar.
HubSpot CRM	Interfaz clara, plan gratuito atractivo, buena experiencia de usuario.	No está especializado en educación y muchas funciones clave son de pago.	Adaptación del pipeline al proceso de matriculación.
Zoho CRM	Buena relación calidad/precio, alto nivel de personalización.	Necesita configuración inicial y curva de aprendizaje.	Plantillas predefinidas y flujo listo para usar.

Funcionalidades implementadas

Alcance real de la versión final



Login y sesiones

Control de acceso a rutas privadas



Leads

Alta pública, alta interna, edición y borrado admin



Panel y Kanban

Estados del embudo y drag and drop



Detalle

Notas, histórico y actividad comercial



Tareas

Seguimientos y objeciones



Dashboard

KPIs, filtros y objetivos del mes

Embudo comercial

El lead avanza por etapas hasta convertirse en alumno o perderse.

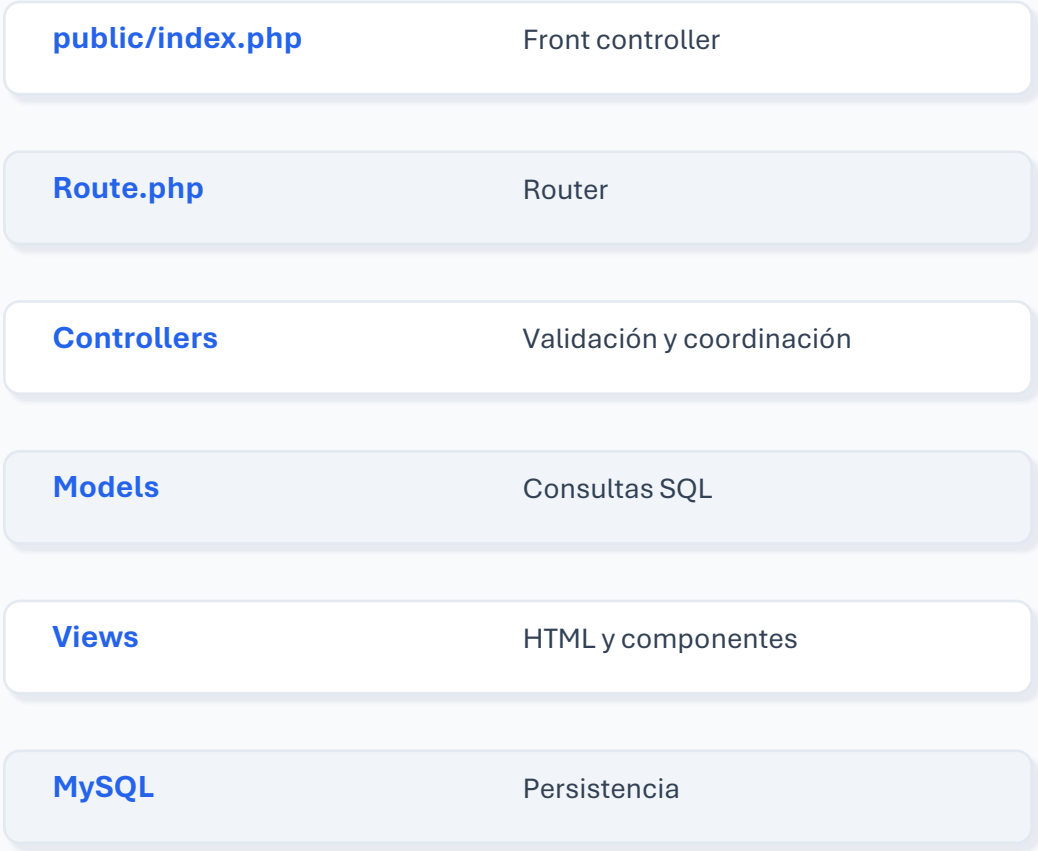


Idea clave

Cada cambio de estado actualiza datos, registra histórico y permite que el equipo vea el progreso comercial en panel, Kanban y dashboard.

Arquitectura final del sistema

PHP MVC con front controller y componentes propios



6. Arquitectura del sistema

La arquitectura definitiva del sistema es una arquitectura web clásica basada en PHP con patrón MVC y front controller. La aplicación no utiliza framework completo; en su lugar, implementa sus propios componentes básicos de configuración, enrutado, sesión, acceso a datos y renderizado de vistas.

El flujo técnico general es el siguiente: el navegador solicita una ruta; public/index.php arranca Composer, configuración y rutas; Route resuelve el controlador; el controlador coordina sesión, validación, modelos y vista; el modelo ejecuta consultas sobre MySQL mediante la clase Database; finalmente se devuelve HTML renderizado al navegador.

6.1 Persistencia de datos

La persistencia se apoya en MySQL mediante la clase lib/Database.php, que encapsula la conexión y ofrece métodos de consulta y actualización basados en sentencias preparadas de MySQLi.

- Conexión configurada mediante constantes BD_HOST, BD_USER, BD_PASS y BD_NAME en lib/db_credentials.php.
- Codificación utf8 para homogeneizar texto y acentos.
- Uso de consultas preparadas en executeQuery() y executeUpdate() para reducir riesgo de inyección SQL.
- Compatibilidad con transacciones (beginTransaction, commit, rollback), aunque en el proyecto final se usan solo puntualmente.
- Modelo de datos relacional con claves foráneas para mantener integridad.

La persistencia está diseñada para un entorno local con XAMPP. Los scripts de migración incluidos permiten reconstruir la base de datos desde cero y cargar usuarios de prueba.

6.2 Procesamiento de información

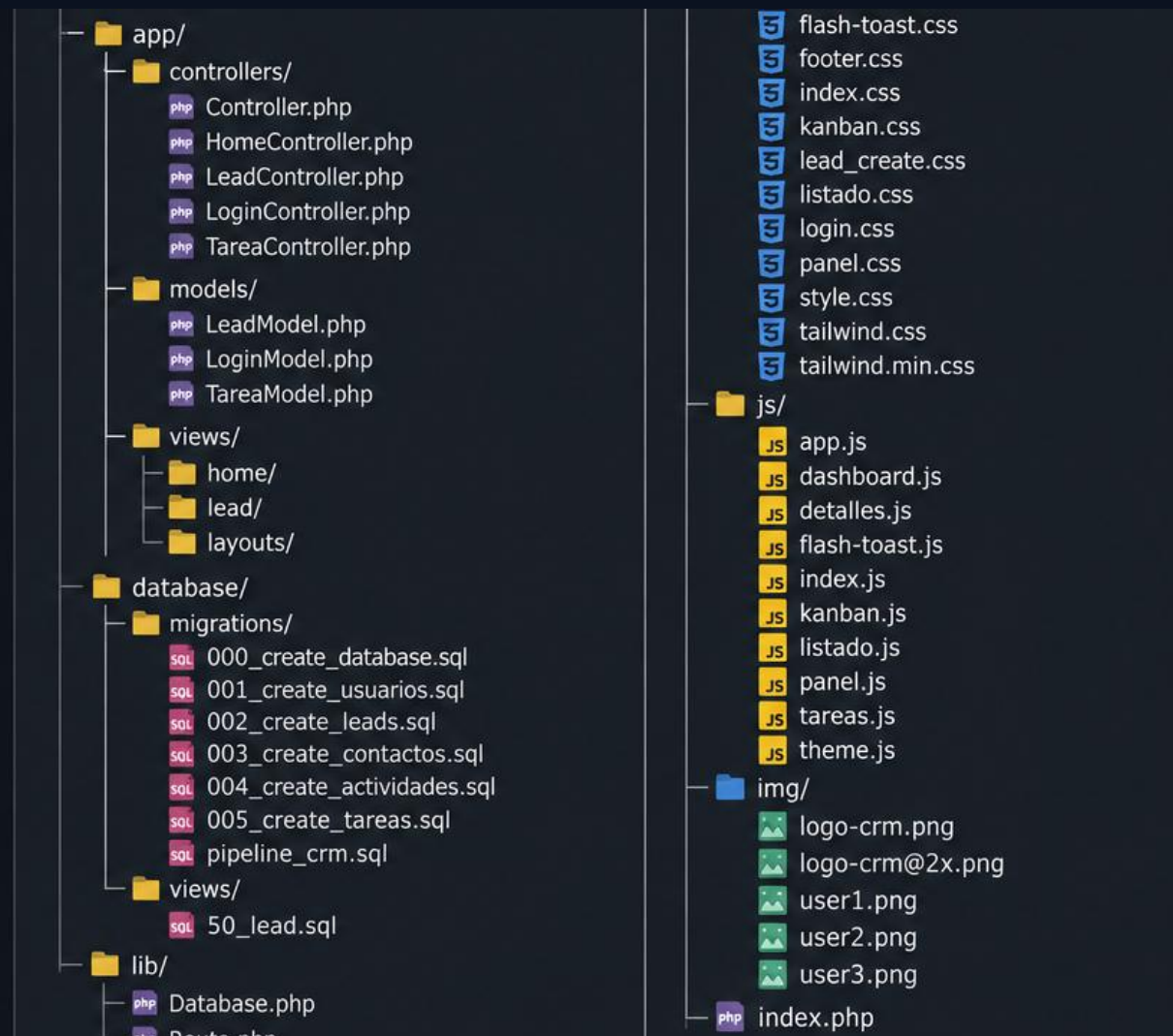
El procesamiento de información reside en la capa backend escrita en PHP. La lógica se distribuye entre controllers y models.

Capa	Archivos principales	Responsabilidad
Front controller	public/index.php	Cargar autoload, configuración y rutas.
Router	lib/Route.php	Mapear URI y método HTTP al controlador correspondiente.
Sesión	lib/SessionManager.php	Control de autenticación y mensajes flash/toast.
Controladores	app/controllers/*.php	Validar entradas, coordinar modelos y decidir la vista o redirección.
Modelos	app/models/*.php	Ejecutar SQL y devolver arrays listos para la vista.
Vistas	app/views/**	Renderizar HTML y componentes visuales.

La solución adopta una separación clara: el modelo no pinta HTML, la vista no ejecuta SQL y el controlador no contiene maquetación. Esta organización facilita el aprendizaje, el mantenimiento y la escalabilidad.

Estructura de carpetas

Organización real del proyecto crm-pipeline



flash-toast.css
footer.css
index.css
kanban.css
lead_create.css
listado.css
login.css
panel.css
style.css
tailwind.css
tailwind.min.css

js/

- app.js
- dashboard.js
- detalles.js
- flash-toast.js
- index.js
- kanban.js
- listado.js
- panel.js
- tareas.js
- theme.js

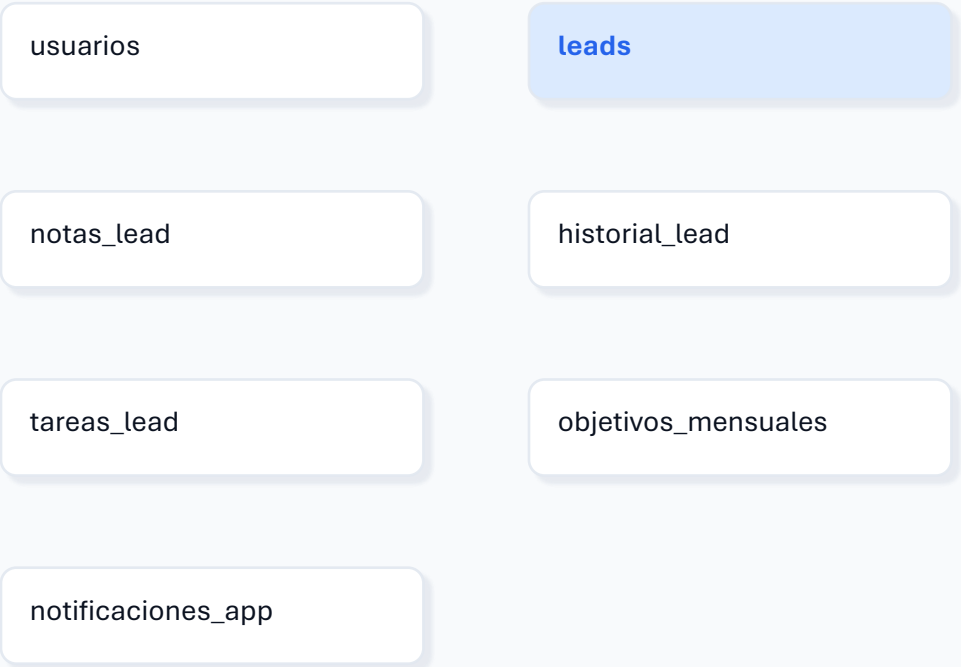
img/

- logo-crm.png
- logo-crm@2x.png
- user1.png
- user2.png
- user3.png

index.php

Modelo de datos

El lead es el centro del sistema



Relaciones destacadas: usuarios 1:N leads, leads 1:N notas, leads 1:N histórico y leads 1:N tareas.

Se incorpora a continuación el diagrama entidad-relación heredado de la documentación previa. Aunque la versión final se ha ajustado al stack PHP y a nombres de tablas definitivos en castellano, la figura sigue siendo útil como apoyo visual para comprender las relaciones generales del sistema.

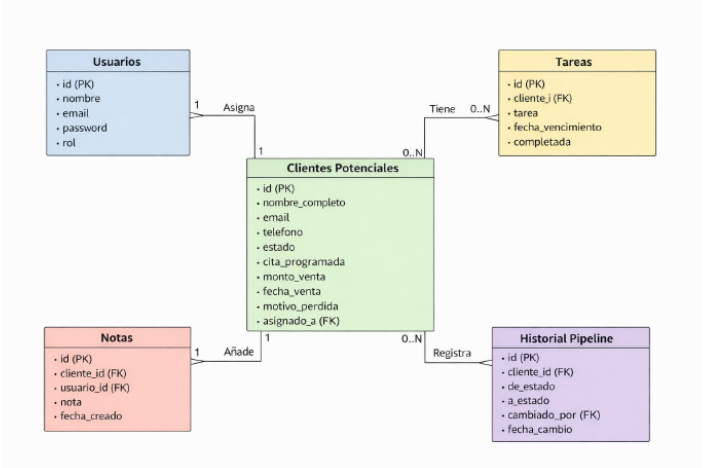


Figura 6. Diagrama entidad-relación procedente de la documentación previa.

4.2 Modelo lógico de datos

El script principal de instalación es database/migrations/pipeline_crm.sql. A continuación se resumen las tablas efectivamente usadas por la aplicación y los campos más importantes.

Tabla	Campos clave
usuarios	id (PK), nombre, email UNIQUE, password_hash, rol, activo, created_at
leads	id (PK), lead_nombre, estado, responsable_id (FK), servicios, indicaciones, lead_score, email, telefono, valor, ultimo_contacto, prioridad, origen, created_at, updated_at
notas_lead	id (PK), lead_id (FK), usuario_id (FK), tipo_actividad, contenido, created_at
historial_lead	id (PK), lead_id (FK), usuario_id (FK nullable), tipo_evento, titulo, descripcion, estado_anterior, estado_nuevo, created_at
tareas_lead	id (PK), lead_id (FK), usuario_creador_id (FK), usuario_asignado_id (FK), tipo_actividad, tipo_bloqueo, solucion_bloqueo, descripcion, fecha_final, estado, leida_asignado, created_at,

Consultas y lógica de acceso a datos

La analítica se resuelve desde modelos con SQL preparado

Autenticación

SELECT usuario por email

Panel

LEFT JOIN leads + usuarios

Dashboard

COUNT, SUM y CASE WHEN

Pipeline

GROUP BY estado

Listado

Filtros dinámicos

Tareas

INNER JOIN leads/usuarios

Patrones clave: consultas preparadas, JOINS, filtros parametrizados, agregaciones, CASE WHEN y control de permisos.

Interfaz de usuario

Vistas principales del CRM



Figura 9. Boceto de la pantalla de login.



Figura 10. Boceto del dashboard / home.

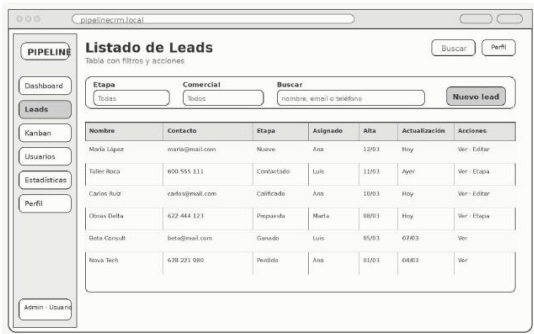


Figura 11. Boceto del listado de leads.

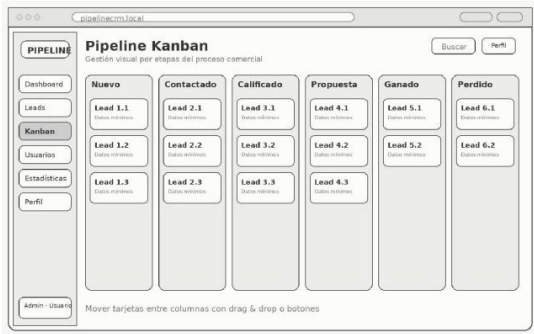


Figura 12. Boceto del tablero Kanban.



Figura 13. Boceto de la pantalla de creación de lead.



Figura 14. Boceto del detalle del lead.

Mapa de navegación

Zona pública

- /
- /login
- /contacto
- /politica-privacidad

Zona privada

- /panel
- /pipeline
- /dashboard
- /leads/nuevo
- /leads/{id}
- /leads/listado
- /tarefas
- /logout

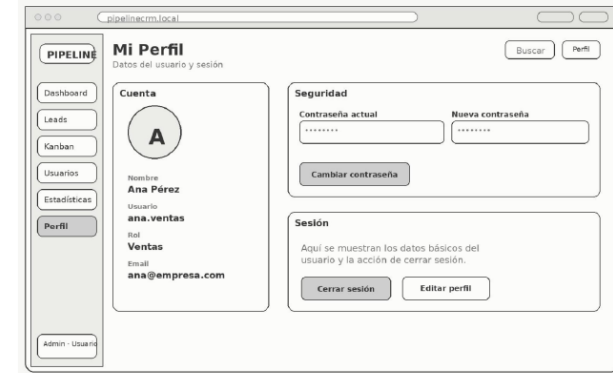


Figura 17. Boceto de perfil / mi cuenta previsto en diseño inicial.

5.2 Mapa de navegación

El flujo de navegación definitivo se organiza en dos áreas: pública y privada.

Zona pública

/ -> portada
/login -> acceso privado
/contacto -> formulario de captación
/politica-privacidad -> página legal

Zona privada

/panel -> panel principal
/pipeline -> kanban del embudo
/dashboard -> análisis y objetivos
/leads/nuevo -> alta interna
/leads/{id} -> detalle del lead
/leads/listado -> listado filtrable
/tarefas -> tareas y objeciones
/logout -> cierre de sesión

Desarrollo por fases

Construcción incremental de la aplicación



0 Base MVC



1 Login y sesiones



2 Leads y panel



**3 Detalle e
histórico**



4 Kanban



**5 Tareas y
objeciones**



6 Dashboard



7 Tailwind y UX

Cada fase añadió funcionalidades y comprobaciones: rutas, sesiones, datos, interacción, automatización y mejora visual.

Aspectos importantes de la implementación

Decisiones técnicas que sostienen el proyecto



MVC claro

Modelo sin HTML, vista sin SQL y controlador como coordinador.



POST/Redirect/GET

Evita reenvíos accidentales de formularios.



SQL preparado

Mejora seguridad y mantiene consultas parametrizadas.



Histórico comercial

Cada actividad o estado importante queda trazado.



Kanban asíncrono

Drag and drop con fetch y persistencia.



Toasts y modo oscuro

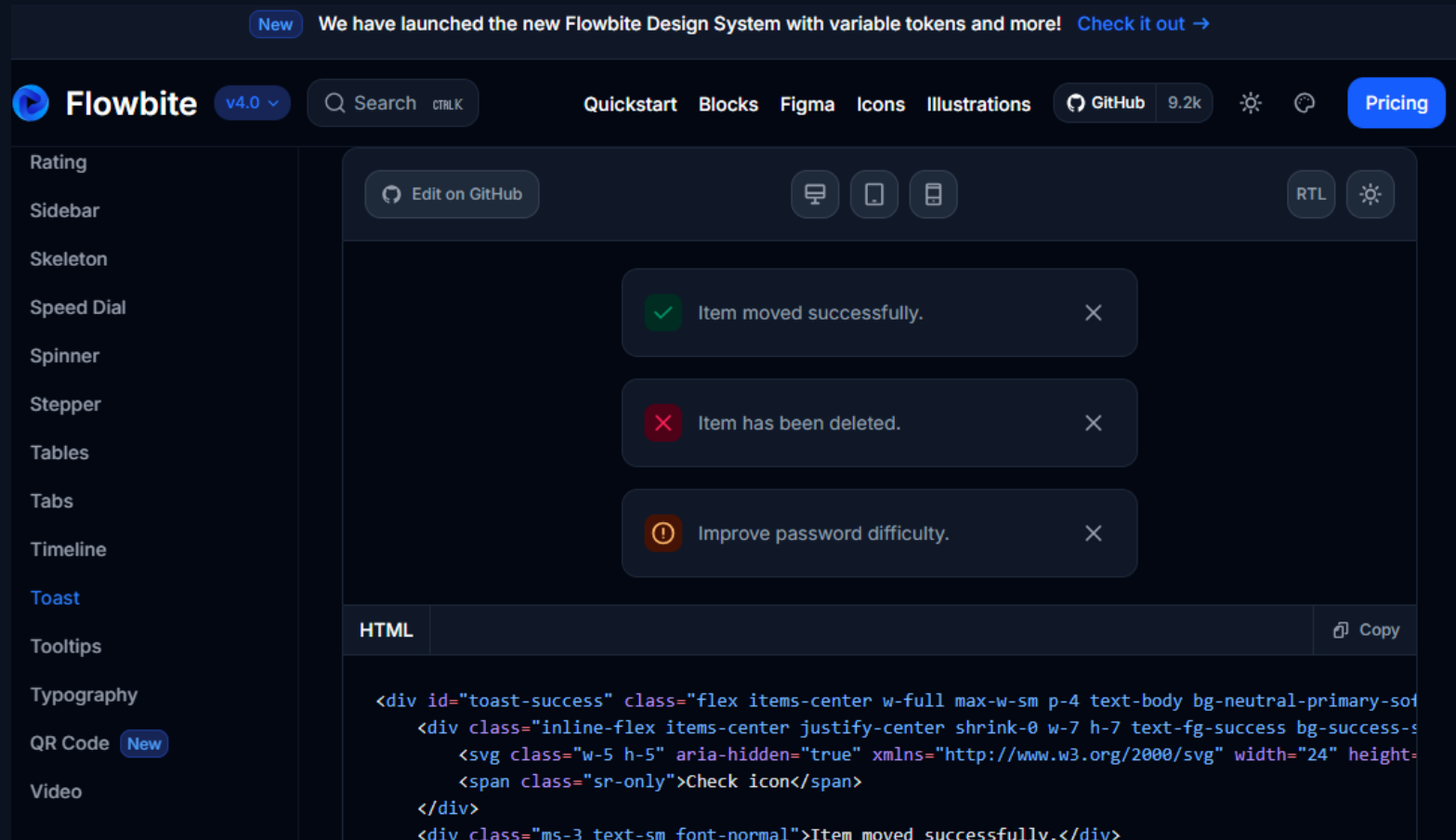
Mejora de experiencia incorporada en fase final.

Dificultad 1

Fase 7 con Tailwind

El reto no fue solo añadir Tailwind, sino integrarlo sin romper CSS previo, layout común, modo oscuro y componentes ya existentes.

- Compatibilizar estilos antiguos y nuevos
- Mantener coherencia visual entre vistas
- Probar toasts, oscuro y objetivos del mes



Dificultad 2

Consultas complejas y relaciones de base de datos

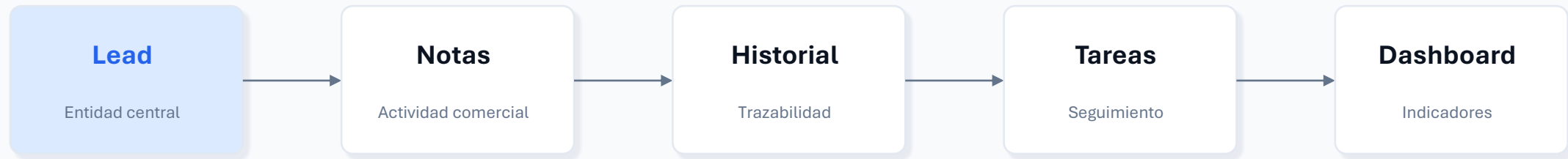
Filtros

Usuario	Desde	Hasta	Servicio	Estado del lead	Origen
Todos ▾	dd/mm/aaaa 📅	dd/mm/aaaa 📅	Todos ▾	Todos ▾	Todos ▾
Aplicar filtros Limpiar					

- El dashboard combina filtros, agregaciones y condiciones por estado.
- Las tareas conectan leads, creador y usuario asignado.
- Las objeciones dependen del estado del lead y generan tareas automáticamente.
- Fue necesario equilibrar claridad del modelo con consultas útiles para la interfaz.

Dificultad 3

Relaciones y sincronización entre módulos



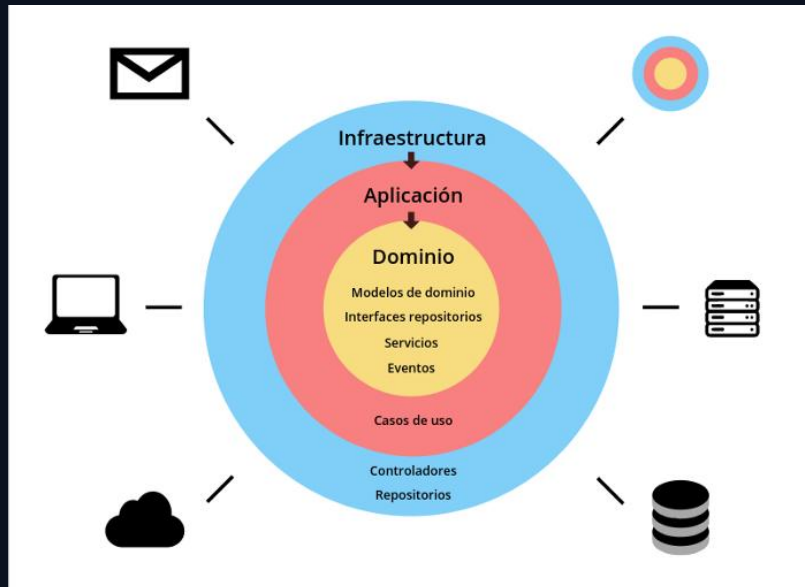
Ejemplo: al cambiar un lead a Objeciones, el sistema debe actualizar estado, crear tarea, registrar histórico y reflejarlo en dashboard.

Cambio respecto a la planificación inicial

De una arquitectura Backend Hexagonal con Java a una aplicación MVC con tecnologías del instituto.

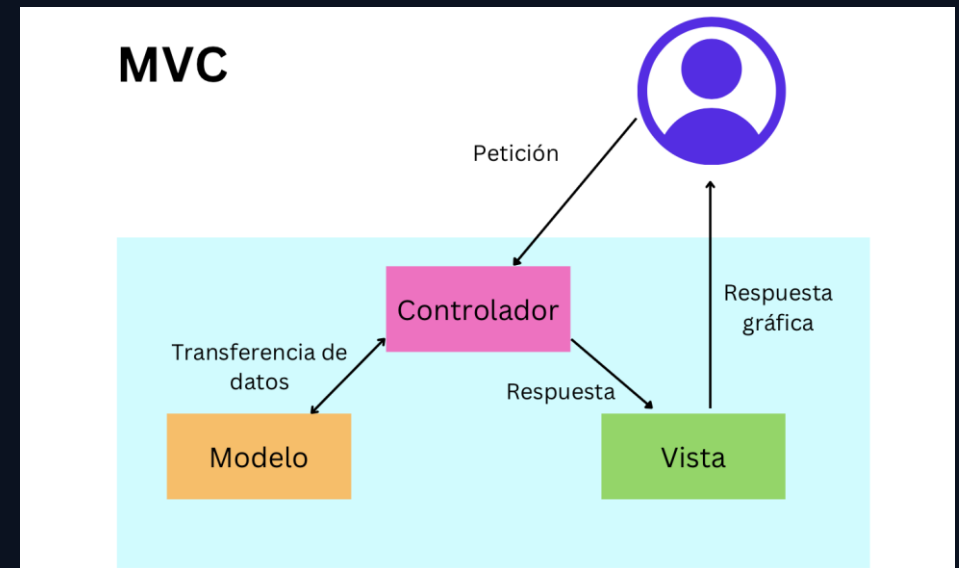
Plan inicial

- Java · OpenLiberty · MicroProfile
- Hexagonal · puertos · adaptadores



Solución final

- PHP · MySQL · JavaScript
- HTML · CSS · Tailwind · MVC



Propuesta de mejora 1

Dar más peso al apartado de Objeciones

Objetivo

- Crear un espacio exclusivo para objeciones.
- Registrar causa, impacto, respuesta propuesta y estado de resolución.
- Convertir obstáculos comerciales en información accionable.
- Relacionar objeciones con etapa, servicio y responsable.

EMBUDO DE VENTAS



Propuesta de mejora 2

Administración de usuarios desde la aplicación

- Crear usuarios desde un panel Admin.
- Asignar roles, activar/desactivar cuentas y resetear contraseña.
- Evitar depender de scripts SQL para usuarios iniciales.
- Mejorar autonomía del cliente y mantenimiento del CRM.



Propuesta de mejora 3

Configuración adaptable al Negocio

Intereses

Cursos o servicios que ofrece la empresa

Etapas

Embudo configurable por cada negocio

Servicios

Catálogo comercial editable

Reglas

Objetivos, prioridades y automatizaciones

Resultado: una aplicación menos rígida, capaz de adaptarse a academias, servicios y procesos comerciales diferentes.

GRACIAS !

PipelineDesk CRM no es solo una práctica técnica: es una solución funcional para organizar un proceso comercial real en centros de formación.



Proyecto completo

Cubre captación, seguimiento, tareas, pipeline y analítica.



Aprendizaje real

Integra PHP, MySQL, JavaScript, CSS, Tailwind y MVC.



Valor funcional

Pensado para centros de formación y procesos comerciales concretos.



Evolución posible

Puede crecer hacia configuración, administración y mejor UX.

PipelineDesk demuestra una aplicación web académica coherente, mantenible y defendible en exposición.